



Agentic AI Engineering Internship 2026

PRE-PROGRAMME PREPARATION GUIDE

Friday 19 June · Sunday 21 June · 2026

Introduction

This document outlines everything you must complete before the CalderR Agentic AI Engineering Internship begins on Monday, 22 June 2026. Two preparation days have been allocated, Friday 19 June and Saturday 20 June, for environment setup, foundational learning, and tool configuration.

Arriving on Day 1 with a fully working environment means the cohort can focus on building from the very first session. Please treat these two days as your first professional obligation within the program.

Objectives

- Configure a complete local development environment
- Install and verify all required software, tools, and libraries
- Obtain and secure API credentials for all required services
- Execute a successful end-to-end LLM call from your local machine
- Establish your GitHub presence and understand version control expectations
- Complete the prerequisite self-assessment before June 22.

Program At a Glance

Detail	Information
Program Start	Monday, 22 June 2026
Prep Period	Friday 19 June – Sundayday 22 June 2026
Commitment During Prep	Self-paced, complete all tasks before Sunday midnight
Weekly Commitment (from Week 1)	4 hours/day · 5 days/week · 20 hours/week
Total Program Duration	10 weeks (200 hours)
Primary Language	Python 3.11+
Primary LLM Provider	Groq (free tier)

Day 1: Friday, 19 June 2026

Day 1 is dedicated entirely to your local development environment, Python setup, version control, and containerization. By the end of this day you should have a working Python environment, your GitHub account configured with SSH access, and Dockers Desktop running.

1. System & Python Environment

Establish a clean, reproducible Python environment using pyenv for version management and virtualenv for project isolation.

Category	Task	Resource / Reference
Python 3.11+	Install pyenv (recommended) and use it to install Python 3.11.9	github.com/pyenv/pyenv
Verify Python	Confirm <code>python --version</code> returns 3.11.x or higher	Terminal: <code>python --version</code>
pip Upgrade	Upgrade pip to the latest version inside your new environment	<code>pip install --upgrade pip</code>
Virtual Environment	Create a project virtual environment named <code>calderr-env</code>	<code>python -m venv</code>
Activate & Verify	Activate the environment and confirm which python points to it	<code>source calderr-env/bin/activate</code>
VS Code	Install VS Code and add the Python, Pylance, and Jupyter extensions	code.visualstudio.com
VS Code Interpreter	Set VS Code to use your <code>calderr-env</code> interpreter via Command Palette	<code>Ctrl+Shift+P</code> → Python: Select Interpreter

Python Packages to Install

With your virtual environment active, install the following core packages. These will be used throughout the 10-week programme.

Package	Purpose
langchain >=0.3	Core AI orchestration framework
langchain-groq >=0.1	Groq LLM integration for LangChain
langchain-community >=0.3	Community integrations and loaders
langgraph >=0.2	Agent workflow and state graph orchestration
groq >=0.11	Official Groq Python SDK
openai >=1.50	OpenAI SDK (used for compatible APIs)
pydantic >=2.0	Data validation and structured output models
python-dotenv >=1.0	Environment variable management from .env files
fastapi >=0.115	Async REST API framework
uvicorn >=0.30	ASGI server for running FastAPI
chromadb >=0.5	Local vector database for RAG systems
sentence-transformers >=3.0	Local embedding models (no API cost)
httpx >=0.27	Async HTTP client for external API calls
rich >=13.0	Beautiful terminal output and formatting
typer >=0.12	CLI application framework
pytest >=8.0	Testing framework
jupyter >=1.0	Notebook environment for experimentation
streamlit >=1.35	Rapid UI framework for AI demos

Tip Save all of the above into a requirements.txt file in your project root. Run `pip install -r requirements.txt` to install everything in one step.

2. Git & GitHub

Professional version control is expected from Day 1. Every project submission, lab, and capstone will be hosted on GitHub.

Category	Task	Resource / Reference
Install Git	Install Git and confirm the version via terminal	git-scm.com · git --version
Global Config	Set your global name and email (must match your GitHub account)	git config --global user.name / user.email
GitHub Account	Create or log in to your GitHub account at github.com	github.com/signup
SSH Key	Generate an Ed25519 SSH key and add the public key to GitHub Settings	ssh-keygen -t ed25519 -C your@email.com
SSH Test	Confirm SSH authentication is working correctly	ssh -T git@github.com
Starter Repo	Create a new public GitHub repository named calderr-ai-2026	github.com → New Repository
Clone Locally	Clone your new repository to your local machine	git clone git@github.com:yourname/calderr-ai-2026.git
First Commit	Add a README.md with your name and a brief bio, then push to main	git add . · git commit · git push
.gitignore	Add a Python .gitignore template to prevent committing secrets or venv	gitignore.io → Python template

Security Rule Never commit API keys, tokens, or passwords to Git. All secrets are stored in a .env file which must be listed in your .gitignore.

3. Docker

All production projects in this programme are containerised. Docker must be installed and verified before Week 1.

Category		Task	Resource / Reference
Install Docker Desktop	Docker	Download and install Docker Desktop for your operating system	docker.com/get-started
Start Daemon	Docker	Launch Docker Desktop and ensure the daemon is running (icon in taskbar)	Docker Desktop UI
Verify Installation		Run the hello-world container to confirm Docker is working	<code>docker run hello-world</code>
Test Container	Python	Run an official Python 3.11 container interactively	<code>docker run -it python:3.11-slim bash</code>
Basic Dockerfile		Write a minimal Dockerfile for a Python script and build it locally	See Dockerfile reference below
docker-compose		Confirm docker compose (v2) is available	<code>docker compose version</code>

Minimal Dockerfile Reference

Create a file named Dockerfile (no extension) with the following content to verify your build process works:

```
FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY . . CMD ["python", "main.py"]
```

Day 2: Saturday, 20 June 2026

Day 2 focuses on API access, LLM fundamentals, your first working AI call, and Jupyter Lab. By the end of this day you should have made a successful LLM call, understand core LLM concepts, and be ready to build from Day 1 of the program.

4. API Access & Credentials

All API keys are stored in a `.env` file at the root of your project. Never share or commit these keys. The `.env` file must be listed in your `.gitignore`.

Category	Task	Resource / Reference
Groq API Key	Sign up at console.groq.com · Navigate to API Keys · Create a new key · Copy it	console.groq.com
HuggingFace Token	Sign up at huggingface.co · Go to Settings → Access Tokens · Create a read token	huggingface.co/settings/tokens
Create .env File	Create a <code>.env</code> file in your project root with both keys	<code>GROQ_API_KEY=gsk_...</code> <code>HF_TOKEN=hf_...</code>
Load & Verify Keys	Use <code>python-dotenv</code> to load and print the first 4 characters of each key	<pre>from dotenv import load_dotenv; load_dotenv()</pre>
Ollama (Optional)	Install Ollama for fully local LLM inference — no API key required	ollama.ai · <code>ollama pull llama3.2</code>

Note Groq's free tier provides approximately 14,400 requests per day with models including `llama-3.3-70b`, `llama-3.1-8b`, and `mixtral-8x7b`. This is sufficient for the entire program.

5. LLM Fundamentals — Required Reading & Viewing

Before writing a single line of AI code, you must understand what you are building on. Complete all resources below before moving to Block 6.

Resource	What to Take Away
Andrej Karpathy: Intro to Large Language Models (YouTube, 1 hr)	How LLMs work: training, inference, tokens, context windows, emergent abilities
Anthropic Prompt Engineering Guide (docs.anthropic.com)	System prompts, user/assistant roles, zero-shot vs few-shot, chain-of-thought
LangChain LCEL Quickstart (python.langchain.com)	The pipe operator , Runnables, prompt → model → parser pattern
Groq API Reference (console.groq.com/docs)	Available models, parameters (temperature, max_tokens, top_p), rate limits
What is an AI Agent? LangChain Blog	The perceive → plan → act → observe loop; tools; memory; planning

Key Concepts to Understand Before June 22:

- Token: the unit of text a model reads and generates
- Context window: the maximum number of tokens a model can process at once
- Temperature: controls randomness; 0 = deterministic, 1+ = creative
- System prompt: the instruction that defines the model's behaviour and persona
- Completion vs Chat: the two API styles; this program uses the Chat format exclusively
- LangChain chain: a composable sequence of steps using the pipe (|) operator
- Agent loop: the cycle an AI agent uses: receive input → reason → act with tools → observe → repeat

6. Your First LLM Call

Follow the steps below to make your first successful call to a Groq-hosted LLM. This is the foundational test that confirms your environment is fully working.

Category	Task	Resource / Reference
Install LangChain + Groq	With your virtual environment active, install langchain and langchain-groq	<code>pip install langchain langchain-groq python-dotenv</code>
Create test_groq.py	Create a new Python file at the root of your project called test_groq.py	<code>touch test_groq.py</code>
Write the Call	Implement the code shown in the reference below — load your key, build a chain, invoke it	See code reference below
Run & Verify	Execute the script; you should receive a coherent response from the model	<code>python test_groq.py</code>
Experiment	Change the model to llama-3.1-8b-instant and compare response speed and quality	<code>model=</code> parameter
Streaming Test	Modify the call to stream the output token-by-token using <code>.stream()</code>	<code>chain.stream(input)</code>

test_groq.py Reference Implementation

```
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()
llm = ChatGroq(model="llama-3.3-70b-versatile", temperature=0.7,
api_key=os.getenv("GROQ_API_KEY"))
prompt = ChatPromptTemplate.from_messages([("system", "You are a helpful AI
engineering assistant."), ("user", "{question}") ])

chain = prompt | llm | StrOutputParser()
response = chain.invoke({"question": "What is an AI agent and why does it matter?"})
print(response)
```

7. Jupyter Lab

Jupyter Lab is the primary experimentation environment used for labs, RAG notebooks, and evaluation work throughout the programme.

Category	Task	Resource Reference
Install Jupyter	Ensure jupyter is installed in your virtual environment	pip install jupyterlab
Launch Jupyter Lab	Start Jupyter Lab from your project root	jupyter lab
Create Notebook	Create a new Python 3 notebook named 01_first_llm_call.ipynb	File → New → Notebook
Reproduce Call	Reproduce your test_groq.py call inside the notebook using separate cells	Run each cell with Shift+Enter
Markdown Cells	Add markdown cells to document what each code cell does	Change cell type to Markdown
Save & Export	Save the notebook and export it as HTML for portfolio documentation	File → Save · File → Export

Self-Assessment

Analyze:

- **Python:** functions, classes, async/await, type hints
- **Git:** branching, committing, pull requests
- **REST APIs:** HTTP methods, JSON, status codes
- **Docker:** images, containers, Compose
- **Large Language Models:** how they work conceptually
- **Prompt engineering:** system prompts, few-shot, CoT
- **LangChain basics:** chains, prompts, output parsers
- **System design:** breaking problems into components
- **Testing & debugging:** pytest, error handling
- Technical writing & documentation

: